Checklist ôn tập đồ án

Nguyễn Hoàng Long | MSSV 25610014

Học theo phần phụ trách để giải thích code, demo nghiệp vụ và thực hiện yêu cầu sửa trực tiếp của giảng viên. Ưu tiên làm chủ luồng xử lý vì vấn đáp cá nhân chiếm 3 trên 10 điểm.

Sáu mảng phụ trách

• Thiết kế database

• Đăng nhập và xác thực

• Tài khoản, hồ sơ và phân quyền

• Đợt đăng ký và quy tắc thời gian

• Nhóm sinh viên, thành viên và trưởng nhóm

• Nộp, quản lý và tải báo cáo

Cách dùng checklist

Đánh dấu khi bạn mở đúng file và hàm, giải thích được dữ liệu đi qua các tầng, demo được tình huống thành công và bị từ chối, rồi tự sửa một yêu cầu nhỏ và chạy lại.

Cách tra đường dẫn file

Các đường dẫn tính từ thư mục gốc project. Tiền tố được rút gọn như sau để dễ đọc:

Tiền tố trong tài liệu	Thư mục thực tế trong project
java/	src/main/java/com/hcmute/topicmanagement/
test/	src/test/java/com/hcmute/topicmanagement/
resources/	src/main/resources/

Nếu một dòng liệt kê nhiều tên file, các tên phía sau nằm trong cùng thư mục với file đầu tiên, trừ khi có ghi đường dẫn khác.

Nguồn đối chiếu: Rubric.pdf; output/Bao_cao_ket_qua_du_an_Group06.docx; mã nguồn project. Ngày đối chiếu: 13/10/2026.

1 Mục tiêu theo rubric

Tiêu chí	Điểm	Cần chuẩn bị
Báo cáo Word	1	Giải thích phần mình làm; bảng phân công, minh chứng file/route, phụ lục cài đặt.
Use-case và CSDL	1	Vai trò, FR/NFR, ERD, PK/FK, kiểu dữ liệu và quan hệ khớp code.
Kiến trúc	1	MVC; ít nhất 3 kỹ thuật Spring; hash mật khẩu; phân quyền.
Chức năng	2	Luồng cơ bản và các ràng buộc nâng cao; hiểu phần còn thiếu.
Thuyết trình và demo	1	Demo live, phân chia thời gian nhóm, chuẩn bị dữ liệu.
Vấn đáp cá nhân	3	Giải thích code mình làm, luồng dữ liệu, annotation; sửa nhanh và chạy lại.
Mở rộng và phản biện	1	Xử lý tình huống biên, xung đột, ngoại lệ, quyền; minh chứng mở rộng.

Ưu tiên ôn vấn đáp cá nhân vì chiếm 3 điểm và yêu cầu làm chủ code của chính mình.

2 Nền tảng chung phải học trước

- ☐ Java dùng trong code: class/interface, kế thừa, constructor, access modifier, enum, record, generic, List/Set/Map, Optional, Stream/filter/map/anyMatch, lambda, Comparator, exception, LocalDateTime, try-with-resources.
- ☐ HTTP: GET/POST/PUT/DELETE; path variable, query parameter, form, JSON, multipart; redirect; mã 200/201/400/401/403/404/503.
- ☐ Spring Boot khởi động ứng dụng, embedded Tomcat và Maven dependency. Đọc pom.xml: Java 21, Spring Boot 4.1.1 và các starter thực tế.
- ☐ Spring MVC: trình duyệt → Security filter → DispatcherServlet → Controller → Service → Repository → MySQL → Model → Thymeleaf → HTML.
- ☐ REST: cùng Service nhưng trả DTO/JSON qua ResponseEntity thay vì tên template.
- ☐ DI/IoC: dependency được Spring cung cấp qua constructor; vì sao không tự new Service/Repository.
- ☐ Tách trách nhiệm: Controller nhận request; Service kiểm tra nghiệp vụ; Repository truy vấn; Entity ánh xạ; Form nhận input; DTO/Summary trả dữ liệu.
- ☐ Validation form khác validation nghiệp vụ; giao diện không thay thế được kiểm tra server.
- ☐ Thymeleaf: th:action, th:object, th:field, th:errors, th:if, th:each; sec:authorize; fragment; flash message và Post/Redirect/Get.
- ☐ Biết dùng debugger, breakpoint, log, đọc stack trace và chạy lại ứng dụng sau thay đổi.

File định hướng

pom.xml

resources/application.properties

java/web/controller/StudentManagementController.java

java/service/UserManagementService.java

java/repository/UserRepository.java.

3 Database và JPA

- ☐ PK, FK, UNIQUE, NOT NULL, index; quan hệ 1-n, n-n, 1-1; vì sao dùng bảng trung gian.
- ☐ Phân biệt ràng buộc database và ràng buộc Service; một UNIQUE(group_id, student_id) không tự ngăn sinh viên vào hai nhóm khác nhau.
- ☐ Giải thích 18 bảng ở mức tổng thể; học sâu users, roles, permissions, user_roles, role_permissions, registration_periods, student_groups, group_members, topic_registrations, reports.
- ☐ users → user_roles → roles → role_permissions → permissions: từ tài khoản đến các quyền cụ thể.
- ☐ student_groups.period_id, leader_id, created_by; group_members nối nhóm với sinh viên; trưởng nhóm đồng thời phải là thành viên.
- ☐ topic_registrations nối nhóm, đề tài, đợt; reports nối đăng ký và người nộp, lưu metadata.
- ☐ @Entity, @Table, @Id, @GeneratedValue, @Column, @ManyToOne, @OneToMany, @ManyToMany, @JoinColumn, @JoinTable, @Enumerated.
- ☐ FetchType.LAZY, join fetch, dữ liệu lazy trong transaction; JpaRepository, derived query, @Query/JPQL và @Param.
- ☐ @Transactional, rollback, readOnly, save và saveAndFlush; DB transaction không tự rollback file trên ổ đĩa.
- ☐ @Lock(PESSIMISTIC_WRITE): hiểu khóa ở thao tác nhóm và đợt; dùng để xử lý yêu cầu đồng thời, không chỉ kiểm tra rồi lưu.
- ☐ spring.jpa.hibernate.ddl-auto=none; database được tạo theo DDL; không tuyên bố Hibernate tự sinh schema trong cấu hình hiện tại.
- ☐ FR là chức năng như tạo nhóm; NFR là đặc tính như kiểm soát truy cập, thời gian phản hồi; Use-case mô tả tương tác, ERD mô tả dữ liệu.
- ☐ Vai trò nghiệp vụ như GVHD/GVPB/thành viên hội đồng có thể được biểu diễn bằng phân công tài nguyên trên tài khoản LECTURER; không nhất thiết mỗi vai trò là một bảng người dùng riêng.

File cần đọc

• database/1.ddl.sql; docs/database-design.md; docs/domain-model.md; docs/authorization-matrix.md.

• java/model/BaseEntity.java.

• java/model/UserEntity.java, RoleEntity.java, PermissionEntity.java, UserRoleEntity.java, RolePermissionEntity.java.

• java/model/RegistrationPeriodEntity.java, StudentGroupEntity.java, TopicRegistrationEntity.java, ReportEntity.java.

• java/repository/UserRepository.java, StudentGroupRepository.java, RegistrationPeriodRepository.java, ReportRepository.java.

Tự kiểm tra

Vẽ được đường nối từ tài khoản sinh viên đến nhóm, đăng ký và báo cáo; giải thích được nơi đảm bảo duy nhất trưởng nhóm và nơi đảm bảo chỉ một nhóm hoạt động trong cùng đợt.

4 Đăng nhập và xác thực

- ☐ Authentication khác Authorization.
- ☐ GET /login chỉ render giao diện; POST /login do Spring Security xử lý, không phải LoginController tự so sánh mật khẩu.
- ☐ formLogin dùng usernameParameter("email"); đăng nhập bằng email_or_code, không dùng MSSV/login_identifier nội bộ trực tiếp cho password login.
- ☐ DaoAuthenticationProvider → DatabaseUserDetailsService.loadUserByUsername → UserRepository.findByEmailIgnoreCase → DatabaseUserPrincipal → PasswordEncoder kiểm tra BCrypt.
- ☐ BCrypt là hash có salt; không giải mã hash để lấy lại mật khẩu. Biết vai trò encode/matches.
- ☐ Tài khoản inactive, chưa đặt mật khẩu hoặc không có role active bị từ chối.
- ☐ Authorities gồm ROLE_STUDENT/ROLE_LECTURER/... và quyền GROUP_MANAGE/REPORT_SUBMIT/...; chỉ lấy các liên kết/role/permission active.
- ☐ Session, JSESSIONID, Authentication/SecurityContext; logout xóa xác thực và hủy session.
- ☐ CSRF token trong form; vì sao POST thiếu token có thể bị 403.
- ☐ Cookie HttpOnly/SameSite/Secure; remember-me và Google OAuth là tùy chọn, không mặc định được bật.
- ☐ Nếu trình bày Google login: đọc thêm ánh xạ email Google sang tài khoản có sẵn và cấu hình client.

File và hàm

• java/config/SecurityConfig.java: passwordEncoder, authenticationProvider, securityFilterChain.
• java/security/DatabaseUserDetailsService.java: loadUserByUsername, toPrincipal.
• java/security/DatabaseUserPrincipal.java.
• java/web/controller/LoginController.java.
• resources/templates/login.html; resources/application.properties.
• Tùy chọn: java/security/GoogleOAuth2UserService.java; java/config/GoogleOAuthConfiguration.java.
• Test tham khảo: test/DatabaseUserDetailsServiceTest.java; SecurityConfigTest.java.

Câu hỏi ôn

1. Vì sao mới tạo tài khoản nhưng chưa đăng nhập được?
2. Ai xử lý POST /login?
3. hasRole('STUDENT') và hasAuthority('REPORT_SUBMIT') khác nhau thế nào?
4. Đăng xuất xóa gì?

5 Tài khoản hồ sơ và phân quyền

- ☐ Luồng tạo SV: nhận tên/MSSV/khoa → kiểm tra MSSV 8 chữ số và trùng → sinh email <MSSV>@student.hcmute.edu.vn → lưu user → gán STUDENT.
- ☐ Luồng tạo GV, kiểm tra email/role; tài khoản mới chưa có password, đặt mật khẩu bằng thao tác riêng.
- ☐ MSSV/login identifier bất biến trong sửa bình thường; không tin field readonly ở trình duyệt là đủ.
- ☐ setPassword và changeOwnPassword: kiểm tra độ dài, xác nhận, hash trước khi lưu; biết luồng thực tế có/không kiểm tra mật khẩu cũ.
- ☐ toggleActive/deleteUser: hiểu các điều kiện bảo vệ tài khoản hiện tại/Admin và xử lý dữ liệu tham chiếu.
- ☐ RBAC: User có Role, Role có Permission; updateRolePermissions/syncPermissions bật/tắt liên kết và thêm quyền mới.
- ☐ Phân quyền URL trong SecurityConfig; @PreAuthorize ở method; kiểm tra sở hữu/nhóm/khoa trong Service; sec:authorize chỉ hỗ trợ giao diện.
- ☐ Quyền thay đổi trong DB không có nghĩa authorities đang giữ trong mọi session được tự nạp lại ngay; đọc nơi principal được tạo để giải thích giới hạn hiện tại.
- ☐ Form validation, BindingResult; redirect sau thành công; flash attribute hiển thị toast.
- ☐ Tìm kiếm/sort/phân trang: hiểu implementation hiện tại lọc/sort danh sách trong bộ nhớ, không mặc định là Pageable/SQL LIMIT.

File và hàm

• java/service/UserManagementService.java: createStudent, createLecturer, updateUserInternal, setPassword, changeOwnPassword, toggleActive, deleteUser, saveRoleAssignments.

• java/service/RoleManagementService.java: updateRolePermissions, syncPermissions.

• java/web/controller/StudentManagementController.java; LecturerManagementController.java; RoleManagementController.java; ProfileController.java.

• java/web/form/UserForm.java; StudentForm.java; LecturerForm.java; PasswordForm.java; PasswordChangeForm.java; PermissionAssignmentForm.java.

• resources/templates/admin/students.html; admin/lecturers.html; admin/roles.html; profile.html; fragments/user-form.html; fragments/user-directory.html; fragments/toast.html.

• resources/static/js/app.js: kiểm tra form phía client và tương tác UI.

• Test tham khảo: test/UserManagementControllerTest.java; RoleManagementControllerTest.java.

Câu hỏi ôn

1. Gõ URL admin trực tiếp có vượt quyền không?
2. Vì sao cần cả Role và Permission?
3. Nếu người dùng sửa hidden field thì server kiểm tra ở đâu?
4. Đổi quyền có tác động ngay tới session cũ không?

6 Đợt đăng ký và quy tắc thời gian

- ☐ Dữ liệu gồm khoảng thời gian giảng viên, khoảng thời gian sinh viên, loại đợt, trạng thái, người tạo; hiểu các mốc chấm/hội đồng cho GRADUATION_THESIS.
- ☐ DRAFT → OPEN → CLOSED → ARCHIVED: không quay ngược/nhảy trạng thái; sửa giữ cùng trạng thái được phép.
- ☐ OPEN chưa đủ; thời điểm hiện tại còn phải nằm trong đúng window.
- ☐ Biên thời gian inclusive: đúng thời điểm bắt đầu/kết thúc vẫn được chấp nhận theo isWithin hiện tại.
- ☐ validateAndNormalize kiểm tra start <= end của từng window, tên và loại; milestone chỉ áp dụng loại thích hợp.
- ☐ Hai giai đoạn rubric: đề xuất/duyệt/công bố đề tài trước, sinh viên đăng ký sau. Nắm phần giao tiếp với module đề tài của bạn khác.
- ☐ requireOpenForLecturer, requireOpenForStudent và inspect là contract dùng chung; server kiểm tra lại, không chỉ ẩn nút.
- ☐ LocalDateTime không chứa timezone; biết thời gian server và dữ liệu cấu hình khi demo.

File và hàm

• java/service/RegistrationPeriodService.java: create, update, validateAndNormalize, requireOpenForLecturer, requireOpenForStudent, isWithin.

• java/model/RegistrationPeriodEntity.java: canTransitionTo.

• java/model/enums/RegistrationPeriodStatus.java; PeriodType.java.

• java/repository/RegistrationPeriodRepository.java.

• java/web/controller/RegistrationPeriodController.java.

• java/web/form/RegistrationPeriodForm.java; resources/templates/faculty/periods.html.

• Đọc nối luồng: java/service/TopicProposalService.java; TopicReviewService.java; TopicPublicationService.java; TopicRegistrationService.java.

• Test tham khảo: test/RegistrationPeriodControllerTest.java.

Câu hỏi ôn

1. OPEN nhưng hết hạn thì sao?
2. Đúng phút kết thúc có được đăng ký?
3. CLOSED có mở lại được không?
4. Hai khoảng thời gian chồng lên nhau thì code hiện tại xử lý thế nào?

7 Nhóm thành viên và trưởng nhóm

- ☐ create: người tạo là trưởng nhóm và được đưa vào members; nhóm gắn một đợt.
- ☐ join: nhóm active, trong window sinh viên, chưa là thành viên, chưa quá số lượng, chưa thuộc nhóm active khác trong cùng đợt.
- ☐ leave: phải là thành viên; trưởng nhóm cần chuyển quyền trước khi rời.
- ☐ transferLeader: người mới là SV active đã thuộc nhóm; nhánh actor STUDENT yêu cầu trưởng nhóm hiện tại; hiểu thêm nhánh quản trị có GROUP_MANAGE.
- ☐ kickMember: chỉ trưởng nhóm, không tự kick mình, người bị kick phải là thành viên hợp lệ.
- ☐ ensureLeaderInvariant: đúng một leader active và leader nằm trong members.
- ☐ ensureNoActiveMembershipInPeriod: phạm vi hiện tại là cùng đợt, không phải cấm vĩnh viễn mọi nhóm ở mọi đợt.
- ☐ findByIdForMutation và findByIdForGroupMutation có PESSIMISTIC_WRITE; giải thích khi hai request cùng join.
- ☐ Transaction, khóa và thứ tự xử lý; không khẳng định chỉ @Transactional đã tự giải quyết mọi race condition/deadlock.
- ☐ Phân biệt nhóm đang tạo có 1-2 người với nhóm đủ điều kiện gửi đăng ký; quy tắc tối thiểu cần đặt đúng bước nghiệp vụ.

File và hàm

• java/service/StudentGroupService.java: MAX_MEMBERS, create, join, leave, transferLeader, kickMember, ensureLeaderInvariant, ensureNoActiveMembershipInPeriod.

• java/web/controller/StudentGroupController.java; StudentGroupRestController.java.

• java/web/dto/StudentGroupRequest.java; StudentGroupLeaderRequest.java.

• java/model/StudentGroupEntity.java.

• java/repository/StudentGroupRepository.java; RegistrationPeriodRepository.java.

• resources/templates/student/groups.html.

• Điểm nối: java/service/TopicRegistrationService.java: submit.

• Test tham khảo: test/StudentGroupControllerTest.java.

Câu hỏi ôn

1. Trưởng nhóm rời nhóm thì sao?
2. Có thể chuyển trưởng nhóm cho người ngoài nhóm không?
3. Hai người tranh chỗ cuối xử lý ở đâu?
4. Một SV có được vào nhóm ở đợt khác không?

8 Nộp quản lý và tải báo cáo

- ☐ Form multipart/form-data và MultipartFile; phân biệt giới hạn mỗi file, tổng request và số lượng file.
- ☐ Upload cần STUDENT và REPORT_SUBMIT, nhưng vẫn phải là trưởng nhóm hiện tại ở Service.
- ☐ Đăng ký phải APPROVED; groupId/registrationId/periodId phải khớp; không tin các ID trình duyệt gửi lên.
- ☐ Kết quả PUBLISHED thì không cho thay đổi báo cáo.
- ☐ File được kiểm tra tên, dung lượng, content type theo cấu hình; chỉ kiểm tra content type chưa phải xác minh toàn bộ nội dung file.
- ☐ Mặc định cấu hình: 10 MiB/file, 10 file/lần, PDF/DOC/DOCX, tổng request 110MB.
- ☐ ReportEntity/MySQL lưu metadata; LocalReportStorage lưu bytes ngoài DB với tên UUID.
- ☐ Vì sao giữ originalName nhưng dùng storedName khác; tránh tên trùng và không dùng đường dẫn người dùng đưa trực tiếp.
- ☐ LocalReportStorage.resolve chuẩn hóa và kiểm tra parent; lưu tạm rồi move; try-with-resources đóng stream.
- ☐ Storage interface và DI: có thể thay implementation mà không đổi toàn bộ nghiệp vụ.
- ☐ saveAllAndFlush thất bại có cleanup file mới; DB transaction không đảm bảo file và DB luôn nguyên tử, đặc biệt xóa file cũ/commit về sau.
- ☐ authorizeView: SV cùng nhóm; GV hướng dẫn/được phân công; Faculty Head cùng khoa; Admin; hiểu kiểm tra quan hệ tài nguyên ngoài authority.
- ☐ Khác biệt SSR redirect/flash và REST 201/400/403/404/503.
- ☐ Không lấy student registration window làm hạn nộp báo cáo khi schema không có deadline báo cáo riêng.

File và hàm

• java/service/ReportService.java: upload, validateFile, ensureCurrentGroupLeader, authorizeView, openDownload, cleanupAfterPersistenceFailure.

• java/service/ReportStorage.java; LocalReportStorage.java.

• java/config/ReportUploadProperties.java; resources/application.properties.

• java/model/ReportEntity.java; java/repository/ReportRepository.java.

• java/web/controller/ReportController.java; ReportRestController.java; ReportAccessRestController.java; ReportViewController.java.

• resources/templates/student/registrations.html.

• Test tham khảo: test/ReportControllerTest.java; ReportServiceStorageFailureTest.java.

Câu hỏi ôn

1. SV thường tự gửi POST có nộp được không?
2. Sửa registrationId sang nhóm khác thì sao?
3. File nằm đâu?
4. DB lỗi sau lưu file thì xử lý gì?
5. Upload sau khi công bố kết quả được không?

9 Annotation phải giải thích được

Nhóm	Annotation/cơ chế	Điều cần giải thích
Bean và DI	@Configuration, @Bean, @Component, @Service, constructor injection, @Value	Bean được tạo ở đâu, dependency/config được đưa vào thế nào.
MVC và REST	@Controller, @RestController, @RequestMapping, @GetMapping, @PostMapping, @PutMapping, @DeleteMapping	HTML khác JSON; URL và HTTP method chọn hàm xử lý.
Input	@RequestParam, @PathVariable, @ModelAttribute, @RequestBody	Dữ liệu lấy từ đâu và bind vào đối tượng nào.
Validation	@Valid, @NotBlank, @NotNull, @Size, @DateTimeFormat, BindingResult	Kiểm tra dữ liệu và phản hồi lỗi trước khi lưu.
Security	@EnableMethodSecurity, @PreAuthorize, hasRole, hasAuthority	Quyền URL, quyền method, và sở hữu tài nguyên.
Giao dịch và truy vấn	@Transactional, @Lock, @Query, @Param	Atomicity của DB, khóa và bind tham số JPQL.
Persistence	@Entity, @Table, @Id, @GeneratedValue, @Column, @ManyToOne, @OneToMany, @ManyToMany, @JoinColumn, @JoinTable, @Enumerated	Ánh xạ entity, khóa và quan hệ.
Exception	@ExceptionHandler	Chuyển lỗi nghiệp vụ thành status/body hoặc thông báo phù hợp.

Không cần học thuộc toàn bộ Spring. Với mỗi annotation đã dùng, phải chỉ được một ví dụ thật và giải thích nếu bỏ nó thì hành vi thay đổi thế nào.

10 Điểm chưa khớp cần hiểu để đáp ứng rubric

1. Nhóm 3-5 SV: StudentGroupService.MAX_MEMBERS hiện là 3; TopicRegistrationService.submit chưa kiểm tra tối thiểu 3. Muốn đáp ứng cần học cách đổi tối đa, cập nhật message/UI/test/seed và kiểm tra tối thiểu ở bước gửi đăng ký. Tạo nhóm ban đầu vẫn có thể cần 1 người để mời thêm thành viên.

2. Không ở hai nhóm: implementation ngăn hai nhóm ACTIVE trong cùng đợt. Rubric không ghi rõ phạm vi đợt, nên cần thống nhất và trình bày rõ trong đặc tả; không tuyên bố đang cấm tất cả các đợt.

3. Hai giai đoạn: validateAndNormalize hiện kiểm tra thứ tự trong từng window, chưa buộc lecturerRegistrationEnd trước studentRegistrationStart. TopicRegistrationService chỉ nhận đề tài PUBLISHED. Cần học kiểm tra quan hệ giữa các mốc và chứng minh kịch bản duyệt/công bố trước khi đăng ký; chỉ có hai window chưa đủ chứng minh quy trình nghiêm ngặt.

4. Email: báo cáo ghi chưa hoàn thành gửi email kết quả; rà soát src và pom.xml chưa thấy JavaMailSender/starter-mail. Nếu bổ sung để đạt mức cao, học SMTP, JavaMailSender, lấy danh sách SV trong nhóm, gửi sau công bố thành công và xử lý gửi lỗi/trùng. Đây là việc phối hợp với người phụ trách công bố kết quả.

5. Deploy: bảng tiến độ ghi 100% nhưng minh chứng nói chưa có URL hosting, trong khi phần bài học ghi biết deploy Render. Cần URL thực tế và kiểm tra truy cập để thống nhất lời trình bày. Có Dockerfile chưa chứng minh đã deploy.

6. Rubric trang nộp bài ghi link deploy không bắt buộc, nhưng bảng chấm lại đưa deploy vào mức Khá/Xuất sắc. Muốn hướng tới các mức đó nên chuẩn bị hosting chạy được; không khẳng định thiếu deploy làm bài nộp không hợp lệ.

7. Các % tự chấm là lời khai trong báo cáo, không thay thế demo và kiểm tra mã nguồn.

11 Luyện sửa code trực tiếp

- ☐ Thay giới hạn tối đa nhóm 3 → 5 và tìm mọi message/UI/test liên quan; thêm kiểm tra đủ 3 ở submit.
- ☐ Thêm kiểm tra hai giai đoạn không chồng lên nhau và thử trường hợp bằng nhau ở biên theo quy tắc nhóm thống nhất.
- ☐ Đổi dung lượng báo cáo, đồng bộ reports.upload.max-size-bytes với spring.servlet.multipart.max-file-size và tổng request.
- ☐ Thêm log tại một bước nghiệp vụ, không log mật khẩu/token/nội dung file.
- ☐ Thêm một điều kiện validation và hiển thị lỗi bằng BindingResult/flash hoặc JSON.
- ☐ Đổi một permission, chứng minh bị chặn qua URL/API dù UI bị sửa.
- ☐ Sửa một tên/field trong template và xác định Model/Form tương ứng.
- ☐ Đọc lỗi runtime và xác định Controller/Service/Repository gây lỗi.

12 Demo và kiểm tra để học

Chuẩn bị Admin, Faculty Head, ba tài khoản SV đã đặt password, một đợt OPEN đúng thời gian, đề tài PUBLISHED và đăng ký APPROVED. Dữ liệu cần khớp trước khi demo nộp báo cáo.

Kịch bản phần Long

1. Login đúng/sai; tạo SV và đặt password; kiểm tra email được sinh.
2. Truy cập sai vai trò để giải thích phân quyền.
3. Tạo/sửa đợt; thử start > end; thử trạng thái không được chuyển.
4. Tạo nhóm, join, thử join trùng/vượt giới hạn, chuyển leader và rời nhóm.
5. Trưởng nhóm nộp báo cáo, thành viên thường bị chặn, thử file sai loại/quá dung lượng.
6. Xem/tải đúng báo cáo; thử tài khoản không liên quan; giải thích phần công bố kết quả nối sang module khác.

Lệnh cần biết chạy từ root repo

```
.\mvnw.cmd spring-boot:run
```

Chạy một test cụ thể, ví dụ:

```
.\mvnw.cmd "-Dtest=StudentGroupControllerTest" test
```

Các test khác cần biết: DatabaseUserDetailsServiceTest, SecurityConfigTest, UserManagementControllerTest, RoleManagementControllerTest, RegistrationPeriodControllerTest, ReportControllerTest và ReportServiceStorageFailureTest.

```
.\mvnw.cmd test
```

```
.\mvnw.cmd package
```

Test dùng cấu hình src/test/resources/application.properties; kết quả test môi trường H2 không tự chứng minh hành vi khóa/tương thích MySQL khi có request đồng thời.

13 Thứ tự học và chuẩn tự đánh giá

1. Java/HTTP/MVC/DI và một luồng tạo SV.
2. Database/JPA/RBAC.
3. Đăng nhập và phân quyền.
4. Nhóm, transaction và khóa.
5. Đợt đăng ký và hai giai đoạn.
6. Upload/download và xử lý lỗi file.
7. Luyện sửa code, demo và nối luồng toàn hệ thống.

Mỗi chức năng chỉ đánh dấu đã nắm khi làm được cả bốn việc: mở đúng file/hàm; giải thích input → kiểm tra → query → output; demo một trường hợp thành công và một trường hợp bị từ chối; tự sửa một yêu cầu nhỏ rồi chạy lại.

Phần ngoài trách nhiệm cần hiểu ở mức kết nối: đề xuất/duyệt/công bố → nhóm/đăng ký → báo cáo → hội đồng 3-5 GV không chấm đề tài mình hướng dẫn → tính điểm → công bố → SV chỉ xem kết quả liên quan. Không cần học sâu mọi hàm của đồng đội trước khi nắm vững sáu mảng của mình.

14 Hồ sơ nộp bài của nhóm

- ☐ Một ZIP theo định dạng thầy yêu cầu: GroupXX_MSSV1_MSSV2_MSSV3_MSSV4_MSSV5.zip; nhóm 06 điền đúng tên nhóm và năm MSSV.
- ☐ Word báo cáo chính, toàn bộ mã nguồn và báo cáo kết quả dự án.
- ☐ Phụ lục cài đặt/config, ảnh minh chứng, bảng phân công chi tiết, file/hàm/route đúng thực tế.
- ☐ Use-case đủ vai trò, FR/NFR, ERD khớp DDL và entity; danh sách cột/kiểu dữ liệu/PK/FK.
- ☐ Liệt kê kỹ thuật Spring đã áp dụng và mục riêng cho tính năng mở rộng để được tính điểm.
- ☐ Nếu công bố deploy: URL chạy thực tế và minh chứng; không chỉ Dockerfile hoặc mô tả kinh nghiệm.
- ☐ PowerPoint không bắt buộc; vẫn phải chuẩn bị thuyết trình/demo và vấn đáp.